

IRONCLAD

TOURNAMENTS

Complete Badge System

Technical & Operational Guide

Canonical registry, server authority, durable
match/game/path evidence, security, and deployment status

Generation date: 2026-08-21
Branch: feature/badge-system-foundation
Commit: daf1455

Repository source of truth. Database runtime validation pending.

Table of Contents

1. Scope and Status	3
2. System Architecture	3
3. Canonical Registry, Artwork, and UI	4
4. Award Persistence and Security Boundary	4
5. Complete Badge Reference	5
6. Authority Families	8
7. Durable Match Authority	9
8. Championship Path Authority	11
9. Flawless Campaign	11
10. Reliable Competitor	12
11. Comeback Commander	13
12. Season Authority	14
13. Live Badge Lifecycle	14
14. Backfill	15
15. Corrections and Invalidation	15
16. Migration Chain	16
17. Testing and Validation	17
18. Preview and Production Safety	17
19. Limitations and Future Work	18

1. Scope and Status

This document describes the complete badge authority implementation present in the repository at commit daf1455. The source of truth is the combination of:

- ? lib/badges/catalog.ts for canonical badge definitions;
- ? lib/badges/authority.ts for server evaluator composition and award writes;
- ? lib/badges/read.ts and lib/badges/dashboard.ts for authenticated reads;
- ? the ordered Supabase migrations under supabase/migrations/;
- ? the tournament, result, leaderboard, profile, Steam, and Relic ELO workflows;
- ? unit and migration-contract tests under tests/.

All 30 canonical badges are represented in the production authority registry.

The migration files are committed to the branch but have not been applied to a database as part of this local work. No production backfill has been run.

The repository currently provides static, mocked, and contract-level validation.

It does not provide proof of live PostgreSQL trigger, RLS, grant, or RPC execution. A disposable local or preview database must be used before deployment.

2. System Architecture

The badge system has six layers:

1. Canonical catalog. lib/badges/catalog.ts defines exactly 30 numbered badges, names, slugs, rarities, unlock meanings, and artwork paths. The database does not duplicate this catalog.
2. Authoritative source data. Profiles, Steam/Relic verification, registrations, tournament brackets, matches, result reports, game evidence, leaderboard events, seasons, and durable authority ledgers remain the source of truth for qualification.
3. Server authority. lib/badges/authority.ts loads authoritative summaries, applies threshold and evidence rules, and writes awards through the trusted Supabase admin client. React and browser code never evaluates or grants badges.
4. Supabase RPCs and durable ledgers. Service-role-only RPCs aggregate historical facts. Durable participant, game, and championship-path tables preserve facts across correction and bracket regeneration.
5. Persistent awards. player_badge_awards stores one append-only award fact per player and canonical slug.
6. Authenticated presentation. Dashboard Server Components read the player's awards with a Clerk-authenticated Supabase client and map rows back to the canonical catalog. Locked badges remain locked because the frontend does not calculate achievements.

The common flow is:

```

Authoritative profile/match/tournament/season operation
-> database state and durable authority are finalized
-> server-side evaluator loads service-role RPC summaries
-> canonical predicate and evidence checks pass
-> conflict-safe player_badge_awards upsert
-> authenticated dashboard read
-> catalog definition + persisted award map to earned/locked UI

```

Badge evaluation is deliberately downstream and best-effort. A badge failure is logged and does not invalidate a successful profile or competition operation.

3. Canonical Registry, Artwork, and UI

lib/badges/catalog.ts exports BADGE_DEFINITIONS, BADGE_TOTAL = 30, and canonical lookup helpers. It also asserts canonical numbering and duplicate-free slugs. The production registry in lib/badges/authority.ts contains the same 30 slugs exactly once:

```

ironclad-recruit, first-deployment, first-victory, battle-tested,
rising-through-the-ranks, first-campaign, iron-regular, tournament-veteran,
season-campaigner, reliable-competitor, five-victories, ten-victories,
twenty-five-victories, iron-streak, unbroken, clean-sweep,
comeback-commander, giant-slayer, giant-hunter, flawless-campaign,
first-advance, semifinalist, finalist, academy-champion, challenge-champion,
elite-champion, double-champion, triple-crown, season-podium, season-champion

```

Each canonical definition resolves artwork through the existing badge asset paths. lib/badges/presentation.ts applies rarity labels and presentation tokens; the dashboard uses the existing showcase and /dashboard/badges collection structure. /dev/badges remains the visual laboratory and is not an authority source.

The production dashboard does not use fixture achievements. loadPlayerBadgeAwards reads only rows for the authenticated player's player_id, validates each slug against the canonical catalog, and maps the result into buildDashboardBadgeData.

4. Award Persistence and Security Boundary

player_badge_awards

The foundation migration creates:

- ? id - stable UUID primary key;
- ? player_id - foreign key to players;
- ? badge_slug - canonical application slug format;
- ? source_type - profile, match, tournament, season, backfill, or admin_correction;
- ? source_id - stable source identifier where available;
- ? source_metadata - JSON object with minimal descriptive evidence;
- ? unlocked_at - database time of award-row creation;
- ? original_unlocked_at - source-event time when reliably available;

? reveal timestamps reserved for later reveal flows;

? created_at.

The database enforces UNIQUE(player_id, badge_slug). The authority layer uses Supabase upsert with onConflict: "player_id,badge_slug" and ignoreDuplicates: true. Retries and concurrent evaluators therefore cannot create duplicate awards.

RLS is enabled and forced. Authenticated users receive only SELECT access, with a policy requiring the player's Clerk JWT subject to match the owning player's clerk_user_id. Insert, update, and delete are unavailable to authenticated users. The service role receives trusted table access and is the only role used by the server authority layer for writes.

Authority RPCs use:

? SECURITY DEFINER where required;

? SET search_path = pg_catalog;

? fully qualified public. references;

? no dynamic SQL;

? execution revoked from public, anon, and authenticated;

? execution granted only to service_role.

Authenticated Supabase clients are intentionally used for dashboard reads. They do not evaluate or grant badges. Service-role credentials remain server-only.

5. Complete Badge Reference

The following entries combine the canonical meaning from catalog.ts with the implemented production predicate, evidence contract, trigger family, backfill behavior, and exclusions.

5.1 Profile and match entry badges

#	Badge / slug	Canonical requirement	Production predicate and evidence
01	IronClad Recruit / ironclad-recruit	Complete identity and ELO verification and become an eligible IronClad player.	profile_completed, Steam identity, current ELO equal to verified Relic ELO, verified faction/division, calculation version, and verification timestamp are all present. source_type=profile; source_id=player_id; original time is the best available Relic verification/profile timestamp.
02	First Deployment / first-deployment	Complete the first official IronClad match.	First qualifying official played match under the existing leaderboard played predicate and valid tournament hierarchy. source_type=match; source is the first match ID; time is the RPC's official-result timestamp or documented fallback.
03	First Victory / first-victory	Win the first official IronClad match.	First distinct qualifying official played victory using finalized winner data and the same tournament exclusions as Badge 02. Match source and first-win time are persisted.

04	Battle Tested / battle-tested	Complete 10 official IronClad matches.	At least 10 distinct qualifying played matches. Cancelled/voided tournaments and outcomes excluded by <code>is_tournament_match_played_for_leaderboard</code> do not count. Source is the tenth match.
05	Rising Through the Ranks / rising-through-the-ranks	Compete successfully in a higher bracket than the player originally entered.	First qualifying completed participation establishes the original bracket; a later qualifying completed participation in a strictly higher academy < challenge < main family awards the badge. Source is the later tournament; time is its authoritative completion time.
06	First Campaign / first-campaign	Complete the first full IronClad tournament.	First distinct qualifying completed tournament participation, not registration-only, withdrawal, no-show, withheld, cancelled, voided, or in-progress history. Source is the first tournament.
07	Iron Regular / iron-regular	Complete 3 IronClad tournaments.	Three distinct qualifying completed tournament participations. Source is the third tournament and its completion time.
08	Tournament Veteran / tournament-veteran	Complete 10 IronClad tournaments.	Ten distinct qualifying completed tournament participations. Source is the tenth tournament and its completion time.
09	Season Campaigner / season-campaigner	Complete at least 4 tournaments in one IronClad season.	Four distinct qualifying completed tournament IDs in one authoritative finalized season. <code>source_type=season</code> ; <code>source_id=season_id</code> ; original time is the fourth tournament completion.
10	Reliable Competitor / reliable-competitor	Complete 10 scheduled matches without a confirmed player-caused no-show.	Best historical run from durable participant outcomes. <code>played</code> and <code>opponent_no_show</code> advance; <code>player_no_show</code> resets; neutral outcomes do neither. Source is the tenth qualifying match and <code>finalized_at</code> .

5.2 Victory milestones

#	Badge / slug	Canonical requirement	Production predicate and evidence
11	Five Victories / five-victories	Win 5 official IronClad matches.	Five distinct qualifying official played victories. Source is the fifth win match.
12	Ten Victories / ten-victories	Win 10 official IronClad matches.	Ten distinct qualifying official played victories. The shared win evaluator cascades Badge 11 and 12 when the threshold is already passed. Source is the tenth win match.
13	Twenty-Five Victories / twenty-five-victories	Win 25 official IronClad matches.	Twenty-five distinct qualifying official played victories. A 25-win evaluation ensures Badges 11, 12, and 13. Source is the twenty-fifth win match.

5.3 Match excellence

#	Badge / slug	Canonical requirement	Production predicate and evidence
14	Iron Streak / iron-streak	Win 3 consecutive played official matches.	Historical best streak of three timestamped played wins. Played losses reset; non-played events are excluded from the played sequence. Source is the third streak match; missing authoritative chronology is excluded.

15	Unbroken / unbroken	Win 5 consecutive played official matches.	Same chronology and sequence rules as Badge 14, threshold five. A later loss does not revoke an earlier achieved streak. Source is the fifth streak match.
16	Clean Sweep / clean-sweep	Win a BO3 2-0 or a BO5 3-0.	Official played result plus durable authoritative final score/game evidence identifying the clean series. 2-1, 3-1, and 3-2 do not qualify. Source is the clean-sweep match.
17	Comeback Commander / comeback-commander	Lose Game 1 and then win the series.	Latest participant outcome must be played; complete durable contiguous game authority must exist; authoritative Game 1 winner must be the opponent; the player must be the official series winner. Source is the match; original time is series finalization.
18	Giant Slayer / giant-slayer	Defeat an opponent whose verified tournament ELO is at least 200 points higher.	Played official win with same-context immutable verified Relic 1v1 registration snapshots and $\text{opponent_elo} - \text{player_elo} \geq 200$, matching calculation version. Source is the upset match.
19	Giant Hunter / giant-hunter	Earn the Giant Slayer achievement three separate times.	Three distinct matches satisfying the exact Badge 18 predicate. Source is the third qualifying upset match.

5.4 Tournament progression and championships

#	Badge / slug	Canonical requirement	Production predicate and evidence
20	Flawless Campaign / flawless-campaign	Win an IronClad tournament without losing a single individual game.	Requires champion authority, latest complete path summary, valid latest path segments, and complete active game authority for every played segment with every game won by the champion. Source is the tournament; original time is <code>first_completed_at</code> . See Chapter 9.
21	First Advance / first-advance	Win the first tournament bracket round.	A genuinely played official match win with authoritative advancement from one bracket round to the next. Bye/default/slot placement alone cannot qualify. Source is the advancing match.
22	Semifinalist / semifinalist	Reach an official IronClad tournament semifinal.	Official bracket progression/round placement in a valid tournament, including legitimate bye semantics where the platform treats the player as reaching the round. Source is the semifinal tournament.
23	Finalist / finalist	Reach an official IronClad tournament final.	Official bracket progression to the final using authoritative round topology, not display labels. Source is the finalist tournament.
24	Academy Champion / academy-champion	Win an official Academy bracket tournament.	Valid completed tournament and authoritative system/recalculation <code>tournament_win</code> for bracket type academy. Source is tournament ID.
25	Challenge Champion / challenge-champion	Win an official Challenge bracket tournament.	Same championship authority with bracket type challenge.
26	Elite Champion / elite-champion	Win an official Main/Elite bracket tournament.	Repository stores the current family as main; the evaluator maps canonical Elite/Main to stored main. Source is tournament ID.

27	Double Champion / double-champion	Win 2 distinct IronClad tournaments.	Two distinct valid completed championship tournament IDs. Recalculation duplicates and multiple brackets do not inflate the count. Source is the second championship tournament.
28	Triple Crown / triple-crown	Win Academy, Challenge, and Elite/Main tournaments at least once each.	At least one valid championship in each stored category family: academy, challenge, and main. Three Main wins alone do not qualify. Source is the threshold championship tournament.

5.5 Season achievements

#	Badge / slug	Canonical requirement	Production predicate and evidence
29	Season Podium / season-podium	Finish an official season in the top 3.	Finalized season authority, not under review, and official final rank <= 3. Tied official ranks follow stored platform semantics. source_type=season; source is season ID; time is finalized_at.
30	Season Champion / season-champion	Finish 1st on the official seasonal leaderboard.	Finalized season, not under review, official rank 1/champion authority from leaderboard_season_champions or equivalent. Ties follow platform authority. Source is season ID and time is finalization.

All 30 slugs are present exactly once in PRODUCTION_BADGE_AUTHORITY_SLUGS.

No catalog slug is missing and no unknown slug is registered.

6. Authority Families

Profile / entry authority: Badge 01

Profile qualification is evaluated from the players row after profile writes, Steam identity synchronization, or Relic ELO verification. The current ELO must equal the verified Relic ELO and all identity, faction, division, calculation version, and verification timestamp fields must be present.

Match participation: Badges 02, 04, and 10

Badges 02 and 04 use official played-match summaries. The underlying SQL joins through tournament_matches -> generated_brackets -> tournament_brackets -> tournaments, rejects cancelled and voided tournaments, and retains the existing is_tournament_match_played_for_leaderboard predicate.

Badge 10 uses the newer durable participant authority rather than inferring attendance from missing no-show rows.

Match victories: Badges 03, 11, 12, and 13

The shared win-count evaluator uses distinct qualifying official victories and threshold data. Threshold jumps cascade lower milestones safely.

Tournament participation: Badges 06, 07, and 08

The tournament summary counts distinct completed tournaments with authoritative

participation facts, valid registration/bracket evidence, and no withheld participation. Registration-only, withdrawn, no-show, cancelled, voided, and in-progress tournaments are excluded.

Bracket progression: Badges 05, 21, 22, and 23

Badge 05 uses first qualifying completed participation as the original bracket and compares later completed participation through academy < challenge < main. Badge 21 requires a played win that proves advancement. Badges 22 and 23 use authoritative round-relative bracket topology and allow legitimate bye semantics where the platform records actual placement in the round.

Match excellence: Badges 14, 15, 16, 17, 18, and 19

The match-excellence RPC provides historical streak, clean-series, and upset evidence. Comeback Commander uses the separate complete-game RPC. Giant Slayer and Giant Hunter share one historical upset predicate and count distinct match IDs.

Championships: Badges 24-28

These use finalized tournament-win leaderboard events, valid tournament state, stored bracket category, and distinct tournament identity. Main is the stored category corresponding to the canonical Main/Elite family.

Flawless Campaign: Badge 20

Badge 20 combines championship authority, durable path summary, participant outcomes, and finalized game authority. It is intentionally evaluated only when the tournament is completed and the necessary path/game facts are already durable.

Seasons: Badges 09, 29, and 30

Season RPCs use repository season identity and finalization facts. Calendar inference is not used. Active or under-review seasons do not award final season badges.

7. Durable Match Authority

Migration 20260820160000_match_authority_foundation.sql adds two append-only, revisioned ledgers.

match_participant_outcome_authority

Each row records a participant-specific terminal fact for a logical match_id + registration_id identity. Important columns include:

- ? tournament_id and registration_id durable foreign keys;
- ? match_id as provenance identity without a foreign key;
- ? outcome_kind;
- ? revision and supersedes_id;

- ? authoritative finalized_at;
- ? source_type, source_id, and JSON provenance;
- ? created_at.

Supported outcomes:

- ? played - positive authoritative evidence of a genuinely played official result;
- ? opponent_no_show - opponent-caused confirmed no-show/walkover outcome;
- ? player_no_show - confirmed player-caused no-show;
- ? double_no_show - both participants failed to attend;
- ? automatic_bye - bracket-generated bye advancement;
- ? admin_default - administrative/default advancement without played evidence;
- ? cancelled - match cancelled;
- ? voided - match/tournament authority voided;
- ? unknown - source evidence cannot safely distinguish the result state.

The absence of a no-show row never creates played. Ambiguous direct admin results remain unknown unless the source contains positive played evidence.

match_game_result_authority

Each row records one finalized game for match_id + game_number. It preserves:

- ? winner and optional loser registration IDs;
- ? series format and finalized game count;
- ? game_authority_complete;
- ? authority state, revision, and supersession;
- ? finalization timestamp and provenance.

Game 1 is authoritative only when its game-numbered evidence is from a trusted modern finalized path. Legacy synthetic game_number = 1 rows are not treated as Game 1 proof. A final BO3/BO5 score alone cannot reconstruct game ordering.

Revision and invalidation

Corrections append a higher revision whose supersedes_id points to the prior fact. Reset and void operations append invalidating/neutral facts. Latest revision selection is deterministic by revision and stable UUID tie-breaker; older authority does not resurface when the latest row is invalidated.

Append functions serialize logical keys with PostgreSQL transaction advisory locks. Durable ledgers do not cascade from tournament_matches or generated_brackets, because those rows can be deleted during bracket recovery or regeneration. This retains tournament and registration identity while keeping replacement match UUIDs distinct.

RLS is enabled and forced on both ledgers. Authenticated reads are limited to a player's own registration identity. Authority mutation is trusted/server-only. Tournament and registration foreign keys use restrictive behavior to preserve history and prevent accidental deletion of factual authority; account privacy flows use the repository's supported anonymization/closure model rather than hard-deleting durable competition evidence.

8. Championship Path Authority

Migration 20260820190000_tournament_championship_path_authority.sql adds:

- ? tournament_championship_path_authority;
- ? tournament_championship_path_summary_authority.

Path segments

A logical segment is identified by:

```
tournament_id + registration_id + path_index
```

path_index is the stable ordered campaign position. round_number and source bracket/round IDs provide topology context; source_match_id is provenance, not a durable foreign key. Supported single-elimination topology is validated conservatively. Malformed or unsupported topology remains incomplete.

Path outcomes mirror durable participant outcomes. played, opponent_no_show, and automatic_bye can remain eligible for a flawless campaign. player_no_show, double_no_show, admin_default, cancelled, voided, and unknown prevent completeness. Empty feeders do not create fake participant segments.

Summary completeness

The summary is a standalone latest-revision contract. It may be complete only when:

- ? the tournament is completed;
- ? current authoritative champion identity matches the registration;
- ? expected path length is known and positive;
- ? observed latest segment count matches expected length;
- ? path indexes are contiguous;
- ? all latest segment revisions are active and allowed;
- ? all required participant authority aligns with each segment;
- ? no segment is missing, unknown, reset, invalidated, or disqualifying.

Segment writes trigger summary recomputation, including late segment arrival after champion finalization. Corrections, resets, regeneration, cancellation, and voiding immediately recompute the latest summary. A previously complete summary cannot remain trusted after the latest path truth becomes incomplete.

Summary and segment revision keys use advisory locking to prevent concurrent revision collisions and stale complete state. A regenerated path must fully rebuild valid replacement segments before completeness can return.

This authority is required for Badge 20 because surviving clean matches alone do not prove that the entire championship path was captured.

9. Flawless Campaign

Canonical meaning: Win an IronClad tournament without losing a single

individual game.

The service-role RPC

`get_player_badge_flawless_campaign_summary(player_id)` combines:

1. completed tournament and system/recalculation `tournament_win` authority;
2. the latest championship path summary;
3. latest path segment revisions;
4. latest participant outcome revisions;
5. latest game revisions for every played path segment.

For every played path segment, all of the following are required:

- ? participant authority is latest played;
- ? a source match exists;
- ? game authority is complete and active;
- ? game numbers are contiguous from Game 1;
- ? finalized game count matches the observed game set;
- ? all game rows agree on series format and finalized count;
- ? every game winner is the champion registration.

The allowed non-played path outcomes are `automatic_bye` and `opponent_no_show`, because neither contains an individual game loss. The following keep the badge locked: player no-show, double no-show, admin default, cancelled, voided, unknown, missing path segments, incomplete path summary, or incomplete game authority.

Examples:

- ? Champion with a complete path and BO3 2-0 / BO5 3-0 played matches: qualifies.
- ? Champion with an automatic bye plus clean played matches: qualifies.
- ? Champion with an opponent no-show plus clean played matches: qualifies.
- ? Any opponent game win in a played series: does not qualify.
- ? BO3 2-1, BO5 3-1, or BO5 3-2: does not qualify.
- ? Final score without durable per-game evidence: does not qualify.
- ? Admin default, double no-show, unknown, or incomplete regenerated path: does not qualify.

If a game is corrected before evaluation, latest authority controls. A corrected opponent win can make the campaign eligible only if the resulting latest complete set is clean; a corrected champion win to opponent win blocks it. Existing badge awards are append-only and are not automatically revoked if facts later change.

Legacy campaigns without durable path and game authority remain locked. No legacy path or final-score reconstruction is performed.

10. Reliable Competitor

Canonical meaning: Complete 10 scheduled matches without a confirmed player-caused no-show.

The reliable competitor RPC reads the latest participant authority per logical `match_id + registration_id`, orders by immutable `finalized_at`, and uses a

stable identifier only as a tie-breaker. Mutable updated_at is not used.

Sequence behavior:

Outcome	Effect
played	Advances the run
opponent_no_show	Advances the run
player_no_show	Resets the run
double_no_show	Neutral; neither advances nor resets
automatic_bye	Neutral
admin_default	Neutral
cancelled	Neutral
voided	Neutral
unknown	Neutral

The evaluator searches for the best historical run, not only the current run.

Ten qualifying outcomes award the badge; a later player no-show does not revoke it. Missing legacy participant authority is ignored rather than interpreted as attendance.

Source evidence is source_type=match, the tenth qualifying match ID, and the participant authority finalized_at.

11. Comeback Commander

Canonical meaning: Lose Game 1 and then win the series.

The evaluator requires:

- ? latest participant outcome is played;
- ? official series authority identifies the player as winner;
- ? complete durable game authority exists;
- ? latest game revisions are active and contiguous;
- ? Game 1 exists authoritatively;
- ? Game 1 winner is the opponent;
- ? the finalized series is complete.

A final 2-1, 3-1, or 3-2 score does not prove Game 1 was lost. Legacy or synthetic Game 1 rows, pending claims, rejected submissions, admin-default, no-show, bye, cancelled, voided, and unknown results do not qualify.

Corrections use latest revisions. A Game 1 loss corrected to a win removes the comeback evidence before award evaluation; a Game 1 win corrected to a loss can qualify if the rest of the complete series proves victory. Invalidated latest authority cannot expose an older active row.

Source is the stable match ID and original time is the official series finalization timestamp, not Game 1 submission time.

12. Season Authority

The season family uses the existing:

- ? leaderboard_seasons;
- ? leaderboard_tournament_season_memberships;
- ? leaderboard_player_season_stats;
- ? leaderboard_season_champions;
- ? leaderboard_point_events.

No calendar-based season is invented.

Season Campaigner - Badge 09

The season must be finalized and valid. Four distinct qualifying completed tournament IDs must belong to the same authoritative season_id. Registration only, withheld participation, cancelled, voided, and incomplete tournaments do not count. Source is the season ID; original time is the fourth tournament's authoritative completion timestamp.

Season Podium - Badge 29

Requires finalized season authority, no active review state, and official final rank ≤ 3 . The evaluator preserves platform tie semantics rather than inventing a tiebreaker. Source time is finalized_at.

Season Champion - Badge 30

Requires finalized, non-review season authority and official champion/rank-1 evidence from the season champion source. Live rank 1 before finalization cannot award. Tied rank-1 behavior follows the stored platform authority.

If a finalized season later enters review, new evaluation skips it. Existing awards are not automatically revoked.

13. Live Badge Lifecycle

Profile and identity hooks

- ? app/profile/actions.ts evaluates Badge 01 after a successful profile write.
- ? app/profile/relic-elo-action.ts evaluates Badge 01 after successful Relic ELO verification.
- ? app/api/steam/callback/route.ts evaluates Badge 01 after successful Steam identity synchronization, including race-recovery paths.

These calls are wrapped in best-effort error handling after the authoritative operation succeeds.

Match hooks

app/tournaments/match-actions.ts and related dashboard/admin actions evaluate match and tournament families after:

- ? confirmed or approved report groups;

- ? admin official result operations;
- ? approved legacy submission review;
- ? finalized match operations.

The evaluator first validates finalized report status or approved submission, then loads authoritative summaries. Durable database triggers write match and game authority in the result transaction where applicable. Badge 20 is only eligible when the downstream tournament evaluation sees completed championship and path authority.

Tournament and season hooks

`evaluateTournamentBadgeAwardsForMatch` loads the completed tournament context. When the tournament is completed, it evaluates tournament participation, progression, championships, Flawless Campaign, and then season evaluation. The season evaluator separately requires finalized season authority. There is no client-side achievement calculation.

Dashboard reads

`app/dashboard/page.tsx` and `app/dashboard/badges/page.tsx` call `buildDashboardBadgeDataFromAwards`. `lib/badges/read.ts` uses the authenticated Supabase client to select the current player's rows. The dashboard never calls the service-role authority client.

14. Backfill

`backfillInitialBadgeAwards` in `lib/badges/authority.ts` is a controlled, server-only function. It loads player IDs, runs the same profile, match, tournament, season, and durable authority evaluators with `evaluationMode: "backfill"`, and returns counts and per-player errors.

The backfill covers all 30 production registry slugs. It is not automatically run on requests and is not exposed as a public route. Database uniqueness makes repeated runs idempotent.

Historical policy is conservative:

- ? missing participant authority does not imply attendance;
- ? missing game authority does not imply Game 1 or clean performance;
- ? missing path summary does not imply a complete campaign;
- ? incomplete or ambiguous season/tournament evidence remains locked;
- ? old mutable or synthetic evidence is not promoted into durable authority.

No production backfill has been executed as part of this branch work.

15. Corrections and Invalidation

Authority ledgers preserve historical revisions and resolve the latest valid fact for evaluation.

- ? Result correction appends a superseding participant/game/path revision.
- ? Match reset appends invalidating or neutral authority.
- ? Tournament cancellation/void invalidates related durable authority without depending on surviving generated match rows.
- ? Bracket regeneration invalidates old path evidence; replacement segments are distinct provenance and must rebuild completeness.
- ? Game invalidation hides the old latest game from consumers.
- ? Championship path summaries refresh whenever latest segment truth changes.
- ? A season entering review prevents new season awards.
- ? Already-issued rows in `player_badge_awards` are not automatically revoked.

This separates audit history from the current qualification projection. It also means operational correction may prevent future awards without silently deleting historical award rows.

16. Migration Chain

All migrations below are additive and must be replayed after the earlier base schema migrations. Do not apply only the badge subset to a clean database.

Migration	Purpose and authority introduced	Badges enabled
20260820100000_badge_award_foundation.sql	player_badge_awards, unique award key, RLS, own-player read policy, match participant/summary RPCs.	01-04 foundation
20260820110000_badge_battch_2_authority.sql	Win threshold, tournament lookup, participant, and tournament summary RPCs.	11-13, 06-08
20260820120000_badge_progression_championship_authority.sql	Tournament authority participant and prestige summary RPCs.	21-28
20260820130000_badge_streak_clean_upset_authority.sql	Match excellence summary RPC.	14-16, 18-19
20260820140000_badge_season_authority.sql	Finalized season, season participant, and season summary RPCs.	09, 29-30
20260820150000_badge_bracket_progression_authority.sql	Bracket progression summary RPC.	05, 21-23
20260820160000_match_authority_foundation.sql	Durable participant/game ledgers, append/read RPCs, match and tournament void triggers, RLS.	Durable foundation for 10, 17, 20
20260820170000_badge_reliable_competitor_authority.sql	Reliable Competitor sequence summary RPC.	10
20260820180000_badge_comeback_commander_authority.sql	Complete-game comeback summary RPC.	17
20260820190000_tournament_championship_path_authority.sql	Durable path and summary ledgers, refresh, completion, regeneration, void, and completion triggers.	Durable foundation for 20
20260820200000_badge_flawless_campaign_authority.sql	Service-role Flawless Campaign summary RPC combining champion, path, participant, and game evidence.	20

All earlier migrations under `supabase/migrations/` are prerequisites for a clean replay because these functions reference the existing tournament, registration, leaderboard, result, ELO, and authentication schema.

No migration has been applied as part of this branch work.

17. Testing and Validation

The repository test strategy has two layers:

Static and mocked validation

- ? Unit tests exercise evaluator composition, threshold cascades, idempotent mocked upserts, dashboard mapping, and source contracts.
- ? Integration tests are migration-contract tests that inspect SQL text, object names, ordering, grants, and security clauses.
- ? Supabase access is mocked; these tests do not execute PostgreSQL triggers, RLS, grants, advisory locks, or SECURITY DEFINER functions.
- ? Regression tests verify that existing badge families remain in the registry and that unsupported badges are not accidentally added.

Application validation recorded for this branch

The repository validation state recorded during implementation was:

```
npm.cmd run lint          PASS
npx.cmd tsc --noEmit      PASS
npm.cmd run test          PASS
npm.cmd run test:integration PASS
npm.cmd run build         PASS
git diff --check          PASS
```

Runtime status

The Badge System migrations have not been runtime-validated against a real local or preview PostgreSQL/Supabase instance as part of this branch workflow. The remaining required validation includes clean migration replay, trigger tests, RLS role tests, SECURITY DEFINER execution, revision concurrency, regeneration durability, controlled fixtures, and a non-production backfill rehearsal.

18. Preview and Production Safety

Frontend Preview alone is insufficient. The target Supabase database must have the complete ordered migration chain before the matching application can safely call the authority RPCs.

Before any Preview runtime test:

1. Prove that Vercel Preview environment variables target an isolated, non-production Supabase project.
2. Confirm the target database has no production users or tournament history.
3. Apply and verify every migration in order in a disposable or approved

staging database.

4. Use controlled fixture identities and tournaments only.
5. Disable or isolate Clerk, Steam, email, webhooks, and other external effects.
6. Deploy the application commit that matches the database schema.
7. Run RLS, RPC, authority, correction, void, regeneration, season, and badge smoke tests.

Never use production users, production tournaments, or production backfill for authority validation. Never assume that a Vercel Preview database is isolated without manually checking its environment-variable mapping and Supabase project identity.

19. Limitations and Future Work

Known limitations proven by the repository:

- ? PostgreSQL runtime validation is pending.
- ? Static migration tests do not prove live trigger or RLS behavior.
- ? Legacy game and attendance evidence can be incomplete; conservative no-award behavior is intentional.
- ? Badge awards are append-only and have no automatic revocation workflow.
- ? Durable authority preserves tournament/registration history and relies on the repository's supported account closure/anonymization model.
- ? Older match-count summary functions retain their documented `official_result_decided_at/updated_at` fallback for legacy match-count ordering; newer streak and durable-authority paths require authoritative timestamps and do not use mutable time as a substitute.
- ? Preview Supabase isolation requires external Vercel and Supabase configuration confirmation; it cannot be established from committed repository files alone.
- ? No Premium entitlement, public-profile badge integration, billing, or reveal mutation flow is implemented in this authority work.

The next operational step is a disposable local or preview PostgreSQL validation of the complete migration chain. That validation must occur before any staging or production migration application or controlled historical backfill.